



Sri Sai PU and Degree College

College Management System

Complete technical and functional documentation — architecture, data model, modules, API reference, deployment, and verified test evidence.

Live deployment sri-sai-pu-and-degree-college.vercel.app

Repository github.com/Username-dev-max/Sri-Sai-pu-and-degree-college

Document date 24 September 2026

Status Deployed and operational

Contents

1	What the system does	3
2	Technology stack	4
3	Architecture blueprint	5
4	Request lifecycle	6
5	Data model	7
6	Roles and permissions	9
7	Design blueprint and screen inventory	10
8	The public website	11
9	Administrator modules	13
10	Faculty, Attendance Staff, Student and Parent	15
11	Authentication and sessions	16
12	API reference	17
13	Deployment	19
14	Performance and caching	20
15	Verification evidence	21
16	Known limits and roadmap	22

How to read this document

Every number in this document was measured against the running system rather than estimated. Where something does not work, or works only within limits, it is stated plainly in section 16 rather than omitted.

One thing to read before going live. Section 16 describes two structural limits that matter at full college scale: concurrent writes can be lost, and the in-memory data model has a size ceiling. Both are measured, and both need the work described there before 1,500 students depend on the system daily.

1 What the system does

The College Management System is a single web application serving two audiences: the public, who read about the college, and the college itself, whose staff, students and parents sign in to do their work. Both halves share one domain, one codebase and one database.

5

roles

75

screens

136

API endpoints

46

database tables

The public website

Open to anyone, no sign-in. Carries the college identity, its departments and programmes, the faculty directory, sports and academic achievements, a photo gallery, and an admissions enquiry form. This is the part that must survive a results-day rush, and it is served from a cache at the network edge rather than by the application on every visit.

The portal

Five roles, each seeing only what belongs to it:

Role	What it does
Administrator	Enrols students, manages faculty and login accounts, builds the academic structure, timetables, fees, notices, announcements and site content. Sees the audit log.
Faculty	Marks attendance for the classes they teach, enters and publishes internal marks, uploads notes and assignments, decides student leave requests.
Attendance Staff	Takes and reviews class registers, contacts the parents of absent students, runs attendance reports. No access to marks or fees.
Student	Sees their own attendance, internal marks, results, fee status and receipts, timetable, notes, assignments and notices. Applies for leave.
Parent	Sees the same for the children linked to their account, and nothing about any other student.

The rule that shapes the design. Nothing is filtered in the browser for privacy. The server decides what a request is allowed to see from the signed-in identity, so a student cannot reach another student's record by editing a URL or a network request. Section 15 records the tests that confirm this.

2 Technology stack

Layer	Technology	Why
Frontend	React 19 · Vite 8 · Tailwind CSS 4	Single-page application; Vite builds content-hashed assets that cache for a year.
Routing	React Router 7	75 client routes, with the server returning the app shell for deep links.
Motion	Framer Motion	Page and component transitions; every loop stops under <code>prefers-reduced-motion</code> .
Charts	Recharts	Dashboard and report visualisations.
Backend	Node 22 · Express 4	27 route modules behind one application.
Database	Supabase PostgreSQL 17	46 tables, 55 foreign keys, 119 indexes.
File storage	Supabase Storage	Two buckets: one public, one private.
Authentication	JWT + server-side sessions	Signed tokens, with a session row that can be revoked so sign-out takes effect immediately.
Passwords	bcrypt	Hashed, never stored or recoverable in plain text.
Documents	pdfkit · docx	Server-generated report and receipt downloads.
Hosting	Vercel	Static site plus one serverless function, one domain.

Repository layout

```
college-management-system/
├── api/index.js           Serverless entry point; wraps the Express app
├── backend/
│   ├── server.js         Express app: mounts 27 routers, exports the app
│   ├── db.js             In-memory database with seed and migrations
│   ├── supabaseStore.js  Reads and writes PostgreSQL
│   ├── sessionStore.js   Sessions and accounts, read live per request
│   ├── fileStore.js      Supabase Storage (public and private buckets)
│   ├── middleware/auth.js Token verification, role checks
│   └── routes/           27 route modules
├── frontend/
│   └── src/
│       ├── pages/        56 page components
│       ├── components/   67 shared components
│       └── index.css      Design tokens and both themes
├── database/
│   └── supabase-schema.sql Generated schema: 46 tables
└── vercel.json           Build, routing and cache policy
```

Scale of the codebase

Measure	Count	Measure	Count
Page components	56	Backend route modules	27

Shared components	67	API endpoints	136
Client routes	75	Database tables	46

3 Architecture blueprint

One Vercel project serves everything. The built React site is delivered as static files from the edge network; the Express API runs as a single serverless function. Because both sit on the same domain, the browser calls a relative `/api` path and no cross-origin configuration is needed.

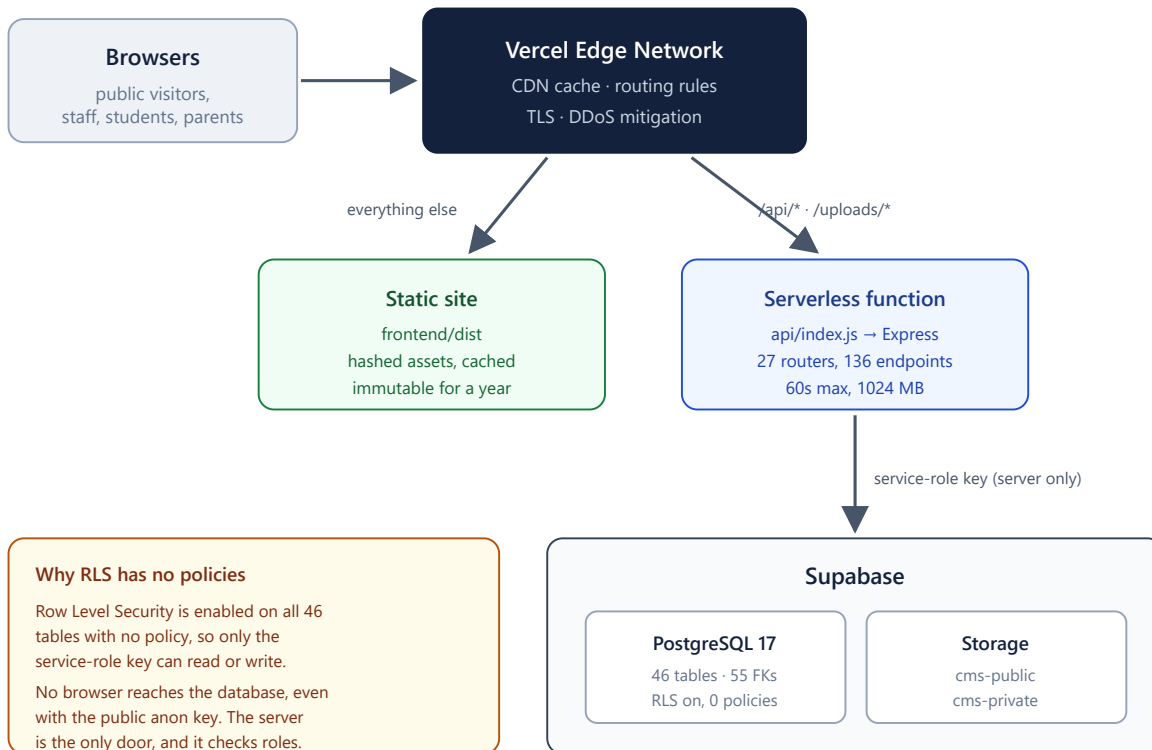


Figure 1 — Deployment topology. One project, one domain, two delivery paths.

Why the API is one function rather than many

The Express application mounts all of its routers under `/api` already. Rather than split it into dozens of independent functions, `api/index.js` re-exports the whole app and the platform routes every `/api` request to it. One function means one warm instance serving every endpoint, so the database is loaded once rather than once per endpoint.

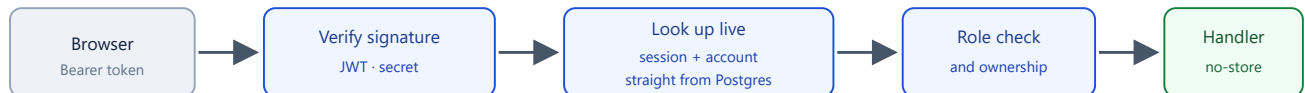
4 Request lifecycle

Two paths matter, and they behave differently on purpose. A public page is read far more often than it changes, so it is cached. A signed-in request carries somebody's private record, so it never is.

A · Public read — the common case



B · Signed-in request — never cached



Why the session is read from the database on every request

Several instances serve traffic at once. A session created on one used to be invisible to the others, so a token issued seconds earlier was rejected with "Your session has ended". Measured on the deployed site: 33 of 40 requests made with one valid token failed. Reading the session live costs one indexed lookup and fixed it — now 40 of 40 pass.

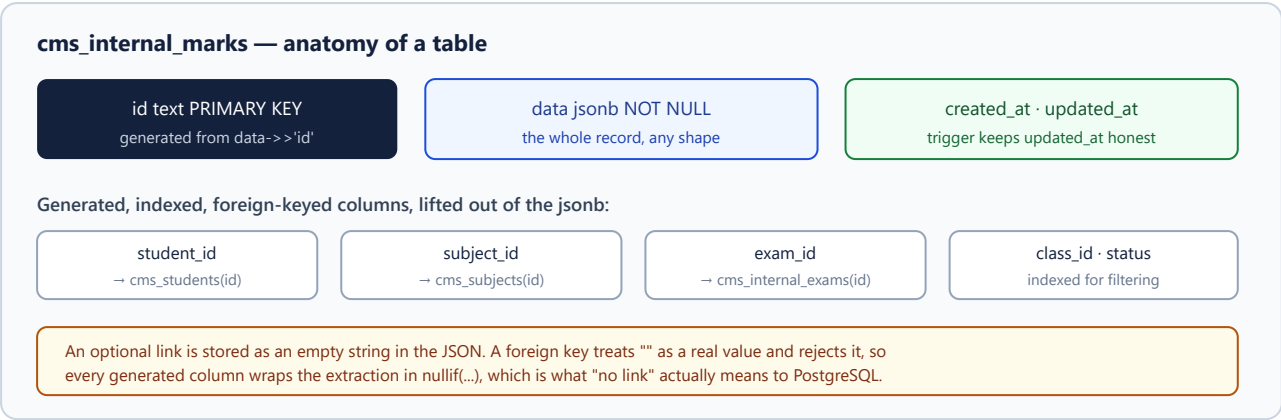
Figure 2 — The two request paths, and the reason authentication reads the database directly.

Cold start

When no instance is warm, the first request pays for one: the function starts and loads the database into memory. That load reads every table. Performed one table after another it took about 15 seconds, longer than a function is allowed to spend starting up, so the first request after a quiet period failed. The reads do not depend on each other and now run together, which brought it to about 3 seconds. Requests arriving during the load wait for it rather than seeing an empty database.

5 Data model

Every collection is its own table with a real primary key, the record held in a `jsonb` column, and generated columns lifted out of that JSON for the fields that carry relationships. Those generated columns are indexed and carry the foreign keys, so PostgreSQL enforces the relationships and the usual queries stay fast, while the record itself keeps whatever shape the application gives it.



The tables

Group	Tables
Academic structure	course_levels, streams, courses, classes, sections, academic_years, subjects, departments
People	students, faculty, users, enrollments, class_teachers, faculty_assignments
Attendance	attendance, attendance_history, attendance_alerts, call_followups
Assessment	internal_exams, internal_marks, internal_mark_history, exams, marks, assignments
Finance	fees, fee_installments, payments, receipts
Teaching	timetable, timetable_publications, notes, leave_requests
Communication	notices, announcements, notifications, admission_inquiries
Public content	gallery_items, sports_achievements, academic_merit, teams, team_members
System	users, auth_sessions, audit_logs, documents, seq, singletons

Two tables that are not shaped like the rest

Table	Shape	Holds
<code>cms_seq</code>	key → value	The counters behind generated record ids.
<code>cms_singletons</code>	key → jsonb	The college profile, the academic policy settings, and the list of migrations already applied.

Current contents

Table	Rows	Table	Rows
Faculty	35	Subjects	24
Departments	12	Courses	11
Students	9	Login accounts	19
Project teams	8	Merit list entries	22

Migration history. The system began on a JSON file. Data was moved to PostgreSQL with every record keeping its original identifier, and the counts were checked table by table afterwards. Uploaded files moved to Supabase Storage at the same time. The original file was never deleted.

6 Roles and permissions

Permission is decided on the server, from the identity in the verified token, on every request. The role selector on the sign-in screen is a convenience; the server rejects a mismatch with the same message as a wrong password, so it cannot be used to discover which role an account has.

Capability	Admin	Faculty	Att. Staff	Student	Parent
Enrol students, edit records	Yes	—	—	—	—
Create and reset login accounts	Yes	—	—	—	—
Manage faculty, departments, subjects	Yes	—	—	—	—
Take attendance	Yes	Own classes	Yes	—	—
Correct a register	Yes	Own classes	Configurable	—	—
Enter internal marks	Yes	Own subjects	—	—	—
Publish marks	Yes	Configurable	—	—	—
Fees, payments, receipts	Yes	—	—	Own only	Own children
View attendance and marks	All	Own classes	Registers	Own only	Own children
Contact a parent about absence	Yes	Yes	Yes	—	—
Decide leave requests	Yes	Yes	—	Apply only	—
Site content and announcements	Yes	Notices	—	—	—
Audit log	Yes	—	—	—	—

What "own only" is enforced by

A student's identity carries the student record it is linked to; a parent's carries the children linked to the account. Requests for anybody else's attendance, marks, fees or documents are refused with **403** — not filtered afterwards, and not trusted from an identifier sent by the browser.

Verified. Cross-account access was tested explicitly for both roles: a student reading another student's fees, marks and attendance, and a parent reading an unlinked student's. Every attempt returned **403**, and every request without a token returned **401**. Section 15 has the full results.

Sessions and sign-out

- Signing in opens a session row; the token carries its identifier.
- Signing out revokes that row, so the token stops working on its next use — on every instance, not just the browser that signed out.
- Changing a password ends every other session for that account.
- Deactivating an account closes its open sessions immediately.
- A new account must replace its temporary password before it can go anywhere.

7 Design blueprint and screen inventory

A note on the design source. The interface is defined in code — design tokens in CSS and React components — rather than in a Figma file. This section is the equivalent blueprint: the layout skeletons, the token system, and the full screen inventory. It describes what actually ships, which a separate design file would not guarantee.

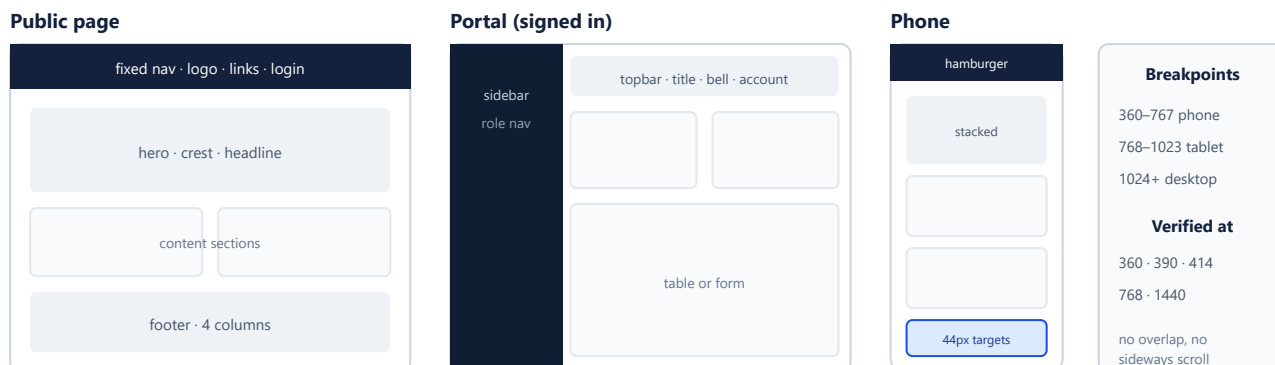


Figure 4 — Layout skeletons and the breakpoints they were measured at.

Design tokens

Colour, surface and text values are CSS custom properties defined once and swapped for the dark theme, so a component never hard-codes a colour that cannot adapt.

Token group	Purpose
Brand navy and gold	College identity: <code>#14213d</code> navy, <code>#c99a3b</code> gold. The navy is replaced with a light variant in dark mode, because it is used as text.
Surface	base, raised, sunken, overlay — the stacking order of backgrounds.
Text	primary, secondary, muted.
Status	success, danger, warning — each with a tinted and a solid variant.
Compatibility layer	The dark theme also remaps Tailwind's neutral ramp, so pages written with ordinary utility classes stay legible without being rewritten.

Typography

- **Source Serif 4** — headings and the college name.
- **Public Sans** — interface and body text.
- System monospace for identifiers and code.

8 The public website

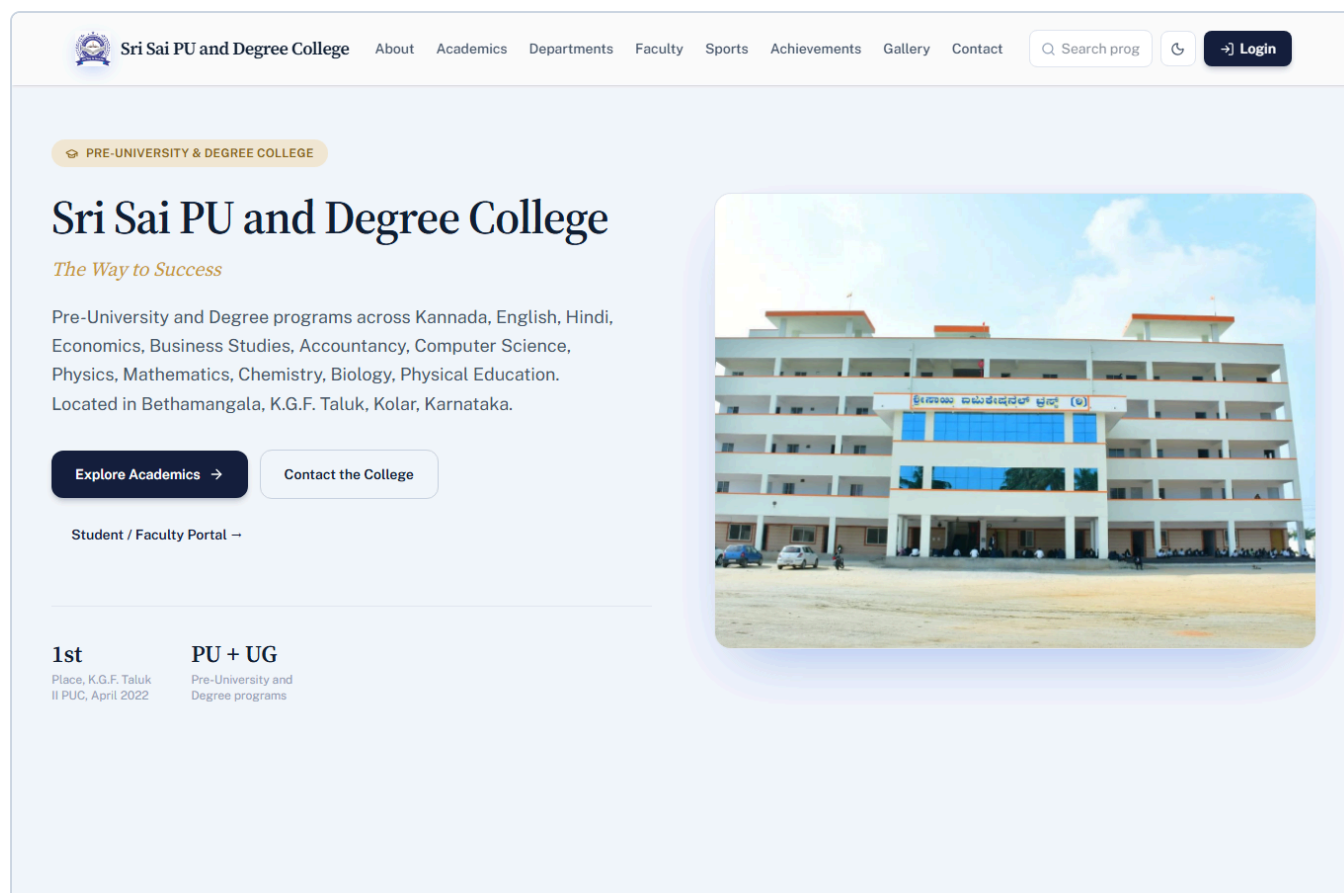


Figure 5 — The home page. Fifteen sections: hero, introduction, programmes, campus, departments, faculty, why choose us, sports, achievements, announcements, gallery, admissions, vision and mission, contact, and the project teams.

Page	Contents
Home	Everything above, assembled from live data rather than fixed copy.
Departments	All 12 subject departments, each with its own page listing its faculty.
Faculty directory	All 35 members, searchable by name and filterable by department, with individual profiles.
Sports	Recorded sporting achievements by year, sport, category and level.
Achievements	The merit list, ranked, with medals for the top three of each group.
Gallery	Photographs by category, with captioned placeholders where none have been uploaded.
Developed By	The eight project teams and their members.

Nothing invented. Where a detail has not been supplied — a founding year, a department vision, a photograph — the page omits it or says plainly that it is not published yet. Editorial placeholders are never shown to visitors.


Sri Sai PU and Degree College

[About](#)
[Academics](#)
[Departments](#)
[Faculty](#)
[Sports](#)
[Achievements](#)
[Gallery](#)
[Contact](#)

FACULTY

Experienced educators across every subject

35 faculty members

All Departments ▾



Murali Krishna

Kannada

8 Years

🎓 D.Ed, B.Ed, M.A

View Profile



Chalapathi H.R

Kannada

15 Years

🎓 M.A, B.Ed

View Profile



Anil Kumar V

Kannada

9 Years

🎓 M.A, B.Ed

View Profile



Shivappa K.N

Kannada

3 Years

🎓 M.A, B.Ed

View Profile



Purnima K.R

Kannada

7 Years

🎓 M.A, B.Ed

View Profile



Tejaswini

English

8 Years

🎓 M.A, B.Ed, English Literature

View Profile



Nagesh R

English

12 Years

🎓 M.A, B.Ed, KSET

View Profile



Vishwanath

English

8 Years

🎓 M.A, B.Ed

View Profile

Figure 6 — The faculty directory, searchable and filterable. Contact details are deliberately not exposed here: the public API returns no email or telephone number for any member.



II PUC ANNUAL EXAMINATION — APRIL 2022

1st Place in K.G.F. Taluk

Sri Sai PU and Degree College secured the top rank in K.G.F. Taluk in the II PUC Annual Examination, April 2022.

Meritorious Students

II PUC Annual Examination — April 2022 · 17 students

Rank	Student Name	Marks
1	Divya	589
2	Kiran	581
3	Bhargavi	580
4	Bhoomi	572
5	Likitha	570
6	Bhavya	564
7	Balaraju	563

Figure 7 — The achievements page. The first three places in each group carry a medal; ranks below that keep an ordinary numeral so the emphasis still means something.

9 Administrator modules

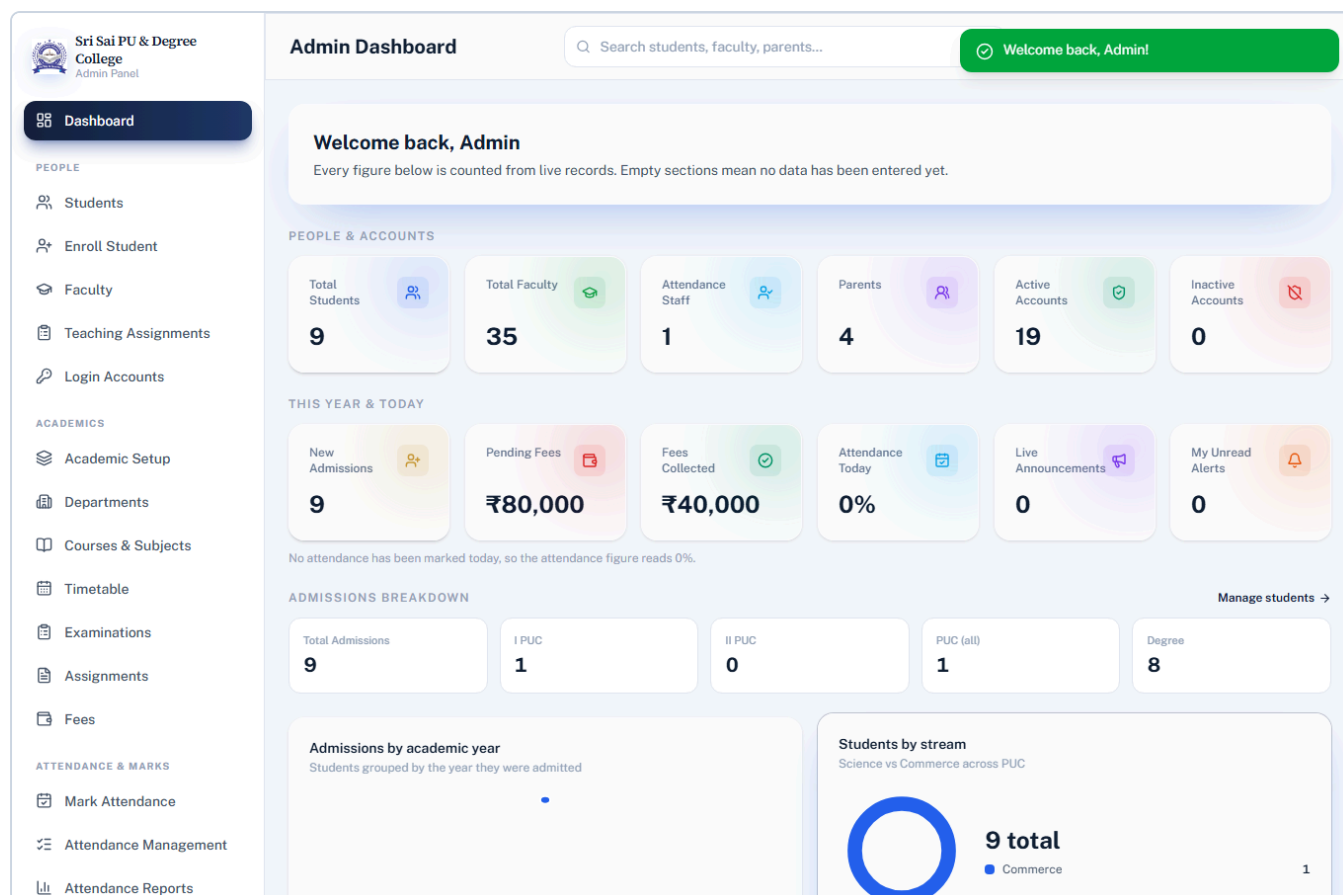


Figure 8 — The administrator dashboard: counts, recent activity and quick access to every management screen.

Module	What it does
Dashboard	Totals, recent activity, and shortcuts into the rest.
Students	The roster, with search, filters and full record editing.
Enrol Student	Admission, with documents; the login account is created as a separate, deliberate step.
Login Accounts	Create, deactivate and reset. A temporary password is shown once and never retrievable.
Faculty	The 35-member roster, with linked login accounts.
Teaching Assignments	Which member teaches which subject to which class — this drives what Faculty can reach.
Academic Setup	Levels, streams, courses, classes, sections and the academic year.
Timetable	Build by class and section, attach a PDF, publish or withdraw.
Internal Marks	Define exams and per-subject maximums; review and publish.
Attendance	Take registers, review every recorded one, run reports.
Fees	Structures, instalments, payments and printable receipts.

Academic Policies	Low-attendance threshold, pass mark, grade scale, who may publish — configurable, not fixed in code.
Site content	College profile, notices, announcements, gallery, sports, merit list, teams, admissions enquiries.
Audit Log	Every administrative action, with who did it and when.



Sri Sai PU & Degree College
Admin Panel

- Dashboard
- PEOPLE
 - Students**
 - Enroll Student
 - Faculty
 - Teaching Assignments
 - Login Accounts
- ACADEMICS
 - Academic Setup
 - Departments
 - Courses & Subjects
 - Timetable
 - Examinations
 - Assignments
 - Fees
- ATTENDANCE & MARKS
 - Mark Attendance
 - Attendance Management
 - Attendance Reports

Student Management

5x

Admin

+ Enroll Student

ID	Name	Department	Course	Sem	Status	
S001	[ADD STUDENT NAME]	DEP07	C04	1	Active	
S202615	AABB	DEP07	C06	1	Active	
S202610	Demo Student 01	DEP07	C06	1	Active	
S202611	Demo Student 02	DEP07	C06	1	Active	
S202612	Demo Student 03	DEP07	C06	1	Active	
S202613	Demo Student 04	DEP07	C06	1	Active	
S20269	Praveen	DEP07	C06	5	Active	
S202624	Priyani c	DEP07	C06	1	Active	
S202614	Ravi	DEP07	C06	1	Active	

Figure 9 — Student management. The same table primitives, search and filter behaviour are reused across every management screen.

Attendance, in the order it is actually taken

1. Academic year, then class, then section, then subject, then date, then period.
2. The roster loads with everyone marked present; the register is corrected from there.
3. Mark-all controls for present and absent.
4. A summary is shown before saving, and a register already taken for that exact slot is refused rather than silently duplicated.
5. A correction requires a reason and is recorded in the history with who changed it.
6. Parents of absent students can be contacted from the register; a call is logged with its outcome.

Internal marks lifecycle

Marks move through **Draft**, **Submitted** and **Published**. Students and parents see nothing until publication. Every change after draft requires a reason and is kept in the history. Ranking uses standard competition order, so two students tied at second are both second and the next is fourth.

10 Faculty, Attendance Staff, Student and Parent

Faculty

Screen	Scope
Dashboard	Today's periods and anything needing attention.
Mark Attendance	Only classes and subjects they are assigned to.
Attendance Management	Their own registers, with correction history.
Internal Marks	Entry for their subjects; class performance where they are class teacher.
Leave Requests	Decide requests from their students.
Notes, Assignments, Timetable	Upload material, set work, see their schedule.

Attendance Staff

A deliberately narrow role: take and review registers, contact parents about absence, run attendance reports. No marks, no fees, no student records beyond what a register needs. Verified: requests for accounts, faculty, fees and the audit log all return 403 .

Student

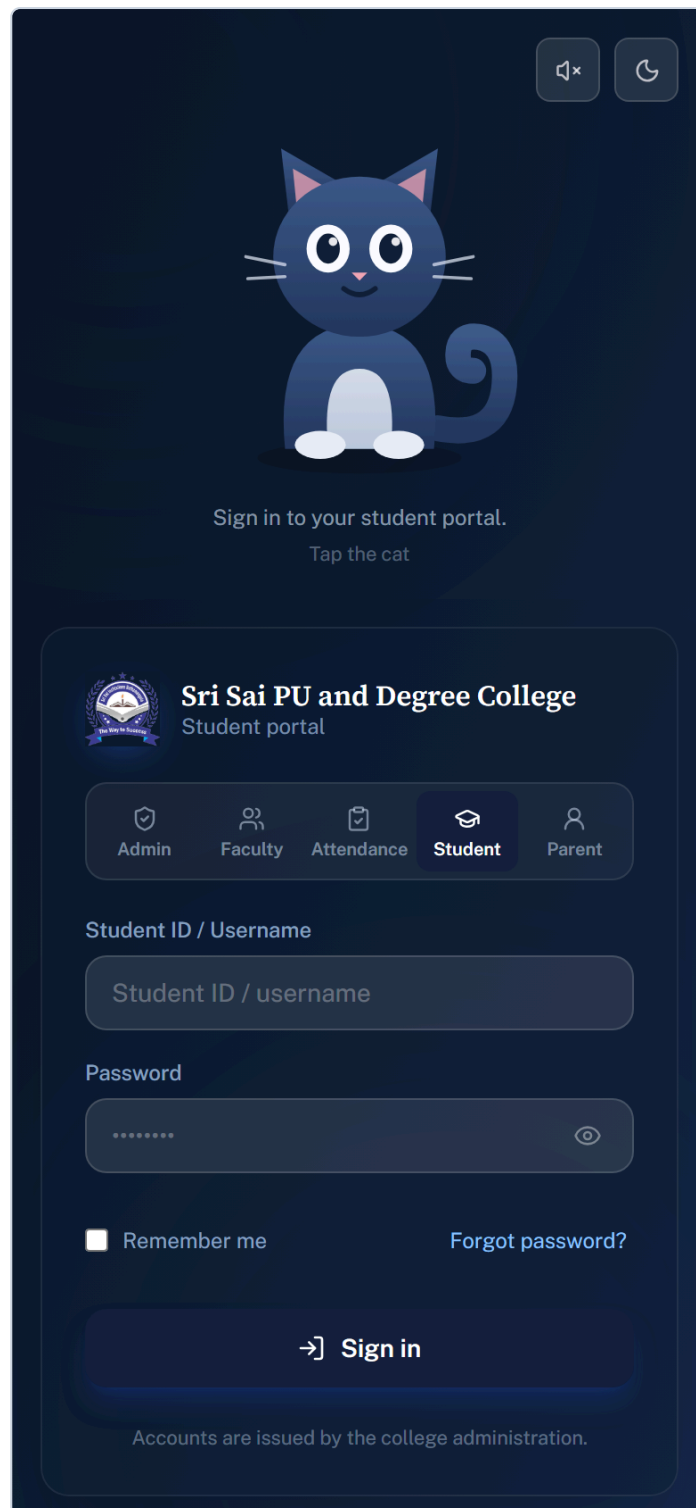
Screen	Shows
Dashboard	Attendance percentage, result standing, fee status, latest notices.
Attendance	Subject-wise and overall, with a calendar view.
Internal Marks	Published marks only, with grade and class position.
Fees	Structure, instalments, payment history, printable receipts.
Leave	Apply with a supporting document; track the decision.
Timetable, Notes, Assignments, Notices	Their class only.

Parent

The same views as their child, for the children linked to the account and no one else. Where a parent has more than one child, a switcher moves between them. An unlinked student's record returns 403 .

Stated plainly. The absence-contact feature records a call and its outcome. It does not place the call. There is no automated or AI calling in this system, and nothing in the interface suggests otherwise.

11 Authentication and sessions



The image shows a mobile app interface for a student sign-in. At the top, there are two icons: a speaker with an 'x' and a circular arrow. Below these is a large, stylized blue cat mascot with white paws and a white belly. The text 'Sign in to your student portal.' is centered below the cat, with 'Tap the cat' written underneath. The main content area is a dark blue rounded rectangle. It starts with the college logo and name 'Sri Sai PU and Degree College' and 'Student portal'. Below this is a horizontal row of five role-based buttons: 'Admin' (shield icon), 'Faculty' (two people icon), 'Attendance' (calendar icon), 'Student' (graduation cap icon, which is highlighted), and 'Parent' (person icon). Under the 'Student' button, there are two input fields: 'Student ID / Username' and 'Password'. The password field has a toggle eye icon. Below the password field are a 'Remember me' checkbox and a 'Forgot password?' link. At the bottom of the form is a large 'Sign in' button with a right-pointing arrow icon. A small note at the very bottom states 'Accounts are issued by the college administration.'

Figure 10 — Student sign-in on a phone. Every role has its own stage and mascot; the cat answers to being petted, and sound stays off until asked for.

How a sign-in works

1. Username and password are checked against a bcrypt hash. A wrong username and a wrong password fail identically, and take the same time, so neither reveals which accounts exist.
2. The chosen role must match the account's. A mismatch returns the same generic message.

3. A session row is written to the database and a signed token carrying its identifier is returned.
4. If the account still holds a temporary password, it must be replaced before anything else.

How a request is checked

1. The token signature and expiry are verified.
2. The session and the account are read *live from the database*, so every instance agrees on whether the session is still open.
3. A revoked session, a deleted account or a deactivated account is refused.
4. Role and ownership are checked against the stored record, not the token, so a change takes effect without signing in again.

Why step 2 reads the database. Sessions used to live only in each instance's memory. With several instances serving traffic, a session created on one was invisible to the others, and a token issued seconds earlier was rejected. Measured on the deployed site: **33 of 40** requests with one valid token failed. After the change, **40 of 40** succeed.

Passwords

- No password appears anywhere in the source code.
- On a new database the first administrator password comes from an environment variable, or is generated, written once to the server log, and must be replaced at first sign-in.
- Demo accounts for the other roles are created only outside production.
- Stored as bcrypt hashes; a forgotten password is reset by an administrator, never recovered.

Before real use. Change the administrator password, deactivate the demo accounts, and rotate the Supabase service-role key and database password.

12 API reference

136 endpoints across 27 route modules, all under `/api`. Everything except the public group and the health check requires a bearer token; most also require a role, and many additionally check ownership.

Public — no authentication

Endpoint	Returns
GET <code>/api/health</code>	Service liveness.
GET <code>/api/public/overview</code>	Everything the home page needs, in one response.
GET <code>/api/public/faculty</code>	The directory. No email or telephone number.
GET <code>/api/public/faculty/:id</code>	One profile, public-safe fields only.
GET <code>/api/public/gallery</code>	Published photographs.
GET <code>/api/public/teams</code> · <code>/teams/:id</code>	The project teams.
POST <code>/api/public/admissions-inquiry</code>	Submit an enquiry.

Authentication

Endpoint	Purpose
POST <code>/api/auth/login</code>	Sign in; opens a session.
GET <code>/api/auth/me</code>	The signed-in identity.
POST <code>/api/auth/logout</code>	Revoke this session everywhere.
POST <code>/api/auth/change-password</code>	Change it; ends every other session.

The rest, by area

Area	Base path	Access
Students	<code>/api/students</code>	Admin writes; own record readable by that student
Faculty	<code>/api/faculty</code>	Admin, Faculty
Login accounts	<code>/api/users</code>	Admin only
Attendance	<code>/api/attendance</code>	Admin, Faculty, Attendance Staff; own for Student and Parent
Internal marks	<code>/api/internal-marks</code>	Admin, Faculty; published only for Student and Parent
Fees and receipts	<code>/api/fees</code>	Admin; own for Student and Parent
Timetable	<code>/api/timetable</code>	Admin writes; all roles read their own

Leave requests	<code>/api/leave-requests</code>	Student applies; Faculty and Admin decide
Notes and assignments	<code>/api/notes</code> · <code>/api/assignments</code>	Faculty and Admin write; class members read
Academic structure	<code>/api/academic-config</code> and others	Admin writes; all read what they need
Policies	<code>/api/settings/academic</code>	Admin writes; all read
Notifications	<code>/api/notifications</code>	Own only
Documents	<code>/api/documents</code>	Owner or Admin; private bucket
Reports	<code>/api/reports</code> · <code>/api/admin/stats</code>	Admin
Audit log	<code>/api/admin/audit</code>	Admin only

Conventions

Status	Meaning
<code>200</code> / <code>201</code>	Succeeded.
<code>400</code>	The request is malformed or fails validation.
<code>401</code>	No token, an invalid one, or a session that has ended.
<code>403</code>	Authenticated, but not allowed this record.
<code>404</code>	No such route, or no such record.
<code>409</code>	A conflict, such as a register already taken for that slot.
<code>503</code>	The database could not be reached. Deliberately not 401: a lookup that failed is not proof a session ended.

13 Deployment

Frontend and backend live in one repository and deploy as one Vercel project.

Configuration

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "frontend/dist",
  "functions": { "api/index.js": { "maxDuration": 60, "memory": 1024 } },
  "rewrites": [
    { "source": "/uploads/(.*)", "destination": "/api/index?__path=/api/public-file/$1" },
    { "source": "/api/(.*)", "destination": "/api/index?__path=/api/$1" },
    { "source": "/((?!api/).*)", "destination": "/index.html" }
  ],
  "headers": [
    { "source": "/assets/(.*)", "headers": [{ "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }] }
  ]
}
```

Decision	Reason
Path carried in <code>__path</code>	A catch-all filename deployed as a single-segment route: <code>/api/health</code> reached the function but <code>/api/auth/login</code> returned 404. Carrying the real path in the destination does not depend on how the platform rewrites a URL.
Backend dependencies at the repository root	The function is built from the root, so that is where its modules come from.
<code>/uploads/*</code> rewritten	Older records store relative image paths; the destination carries the filename itself.
SPA fallback excludes <code>/api/</code>	So an unknown API path returns JSON <code>404</code> rather than the HTML page.
<code>maxDuration: 60</code>	Headroom for a cold start that must load the database first.

Setting it up

1. Push the repository to GitHub.
2. In Vercel, import it. Leave **Root Directory** at the repository root — not `frontend`, or the configuration and the API are never seen.
3. Choose a preset of **Other**; `vercel.json` supplies the build settings.
4. Add the environment variables below, then deploy.

Variable	Required	Value
<code>SUPABASE_URL</code>	Yes	The project URL.
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Yes	Server only. It bypasses Row Level Security, so it must never appear in frontend code, a <code>VITE_</code> variable, or the repository.
<code>JWT_SECRET</code>	Yes	A long random value. The server refuses to start in production without it.

<code>NODE_ENV</code>	Set by Vercel	<code>production</code> .
<code>PUBLIC_CACHE_SECONDS</code>	No	Edge cache lifetime for public pages; default 60, 0 disables.
<code>DATA_DIR</code>	Do not set	Local file storage only. On Vercel it would point at a disk that does not persist.

Running it locally

```
npm run install:all    # root, frontend and backend
npm start              # backend → http://localhost:5000
npm run dev:web        # frontend → http://localhost:5173
```

Open `http://localhost:5173` . Vite binds IPv6 localhost, so use `localhost` rather than `127.0.0.1` .

14 Performance and caching

What changed, and what it was measured at

Measure	Before	After	How
Database load at startup	14.7 s	3.1 s	45 table reads issued together instead of one after another
Backend ready	~40 s	4.7 s	the same change
Artwork total	1061 KB	513 KB	re-encoded for their real display size
College crest	283 KB	30 KB	512px shipped for a 42px slot
Sign-in page weight	461 KB	208 KB	mostly the crest
Sign-in page settled	3.5–5.3 s	2.1–2.5 s	measured in a browser
Public API caching	none	edge cached	<code>s-maxage</code> with background refresh

The caching policy

What	Policy	Why
Public API reads	<code>s-maxage=60</code> <code>stale-while-revalidate=300</code>	Read constantly, changed rarely. The edge answers repeat visitors; an edit appears within a minute.
Signed-in responses	<code>no-store, private</code>	Someone's private record must never sit in a shared cache.
Build output	<code>max-age=31536000, immutable</code>	Filenames are content-hashed, so a change is a new filename.
Campus artwork	a day in the browser, a week at the edge	Changes rarely, and only deliberately.
Uploaded files	an hour in the browser, a day at the edge	Served through the function from Storage.

Verified live. Three consecutive requests to the overview endpoint: `MISS`, then `HIT`, then `HIT`, with the age climbing. At tens of thousands of daily readers this is the difference between one function execution per visitor and a few hundred a day.

Clearing the cache

There is no button to press. A public page refreshes itself within `PUBLIC_CACHE_SECONDS`, sixty seconds by default; lower it for a site edited often, raise it for more headroom under load. Signed-in data is never cached,

so an administrator's change is visible to that administrator immediately. A deployment clears everything.

15 Verification evidence

Every figure below came from running the system, not from reading the code.

What was checked	Result	Covering
Browser audit, production build	967 / 967	Every public page and all 61 role routes, desktop and mobile
Routing and deployment rules	17 / 17	Deep links, assets, API, auth, 401 and 404, uploads
Multi-user sessions	40 / 40	One token across many instances (was 7 of 40)
Cross-instance session validity	5 / 5	Two separate processes; sign-out propagates
Cache policy	11 / 11	Public cached, signed-in never, writes never
Contrast, both themes	10 / 10	Above the 4.5:1 AA threshold
Mobile layout	9 / 9 pages	360, 390 and 414px: no overlap, no sideways scroll
Fresh clone	6 / 6	Cloned from GitHub, installed, built, served real data
Database persistence	9 / 9	Written by one process, read by another after restart
Sign-in, all five roles	5 / 5	Real browser against the live site

Access control, tested rather than assumed

Attempt	Result
Student reads another student's fees, marks, attendance	403
Parent reads an unlinked student's record	403
Faculty lists accounts or reads the audit log	403
Attendance Staff lists faculty or fees	403
Any protected endpoint without a token	401
A token used after signing out	401
A deactivated account signing in	refused
Anonymous fetch of a private stored file	blocked
Public faculty directory exposing contact details	none exposed

Repository hygiene

- No secret appears in any tracked file, or anywhere in the commit history.
- The runtime database, uploads, environment files and backups are excluded from version control.

- A fresh clone builds and serves with only the environment variables supplied.

16 Known limits and roadmap

This section is the honest part of the document. Everything here is measured, and everything here matters before the whole college depends on the system daily.

1 • Concurrent writes can be lost

Each instance holds the database in memory and writes back what changed against the copy it started with. Two instances writing in the same moment can overwrite one another. Tested directly with two processes writing at once:

```
records written by A and B that survived: 1 of 2
>>> ONE WRITE WAS LOST
```

What it means in practice. Two teachers saving registers in the same minute, or two office staff recording payments together, can lose one of them silently. **Severity: high.**

2 • The in-memory model has a size ceiling

Every instance loads the whole database at startup. Today that is 542 rows and about 179 KB, which is nothing. At full scale it is not:

Projection	Figure
Students	1,500
Attendance rows per year (6 periods × 200 days)	~1.8 million
Extrapolated size, attendance alone	~581 MB
Memory available to the function	1,024 MB

Startup would exceed the time limit long before it exceeded the memory limit. **Severity: high, on a timescale of one academic year.**

The fix for both

They share a cause and a remedy: **query PostgreSQL per request rather than holding the database in memory.** That removes the lost writes, the memory ceiling and the staleness between instances together. It rewrites the persistence contract and touches all 27 route modules, so it is a project rather than a patch, and is best done module by module with the existing tests run at each step.

Smaller items

Item	Severity	Note
Data edits can lag between instances	Medium	Same cause. Sessions are already read live and unaffected.
No rate limiting on sign-in	Medium	Add it in the application or at the firewall.
Faculty can read every student record	Medium	Narrow to their own classes.

Announcement and gallery uploads are public by URL	Low	Do not put private files through those screens.
Upload size limited to 4.5 MB	Low	A platform limit on the request body.
Demo accounts still present	High until done	Deactivate them and change the administrator password.
Credentials shared during development	High until done	Rotate the service-role key and database password.

Suggested order

1. **Now.** Change the administrator password, deactivate demo accounts, rotate the Supabase credentials.
2. **Before full rollout.** Move persistence to per-request queries (items 1 and 2).
3. **Alongside.** Rate limiting; narrow Faculty's view of student records.
4. **As the college grows.** Review the hosting plan: a results-day rush from one town resembles the traffic pattern that automatic mitigation is designed to challenge.

What this system does well today. Five roles with server-enforced boundaries that were tested rather than assumed; a public site that reads from live data and is cached to survive a crowd; attendance and marks workflows built the way the work is actually done; and a deployment that a fresh clone reproduces. The limits above are about scale and concurrency, not correctness.